Department of Information and Communication Technology
Faculty of Science and Technology
Bangladesh University of Professionals

BUP Research Bridge

Supervised by:

Nazneen Akter
Assistant Professor
Department of ICT, FST
Bangladesh University of Professionals

Suman Saha
Assistant Professor
Department of IT&RE
University of Frontier Technology, Bangladesh

Prepared by:

S.M. Nabil Mashrafi
ID: 23549009080
BICE-2023

Md Delwar Husen
ID: 24549010076
BICE-2024

Mohammad Shahriar Shefat
ID: 24549010080
BICE-2024

Nabil Hasan Evan
ID: 24549010082
BICE-2024

Date of Submission: 12-10-2026

BUP Research Bridge

1. Architecture & Justify:

Selected Architecture: Three-Tier Architecture

For BUP Research Bridge, we have selected Three-Tier Architecture as the system architecture. This architecture divides the system into three logically independent layers: the Presentation Layer, the Application Layer, and the Database Layer. Each layer has its own responsibility. The separation makes the platform easier to maintain, modify, test, and extend.

BUP Research Bridge is a web-based research platform for the Computer Science and Engineering (CSE) and Information and Communication Technology (ICT) departments of Bangladesh University of Professionals. Students in these departments often have to look through departmental pages, personal profiles, and informal channels to find a supervisor, a thesis opportunity, a faculty research interest, a previous thesis, or an alumnus with relevant experience. The platform gathers that information in one place.

The system serves four roles: student, faculty, alumni, and administrator. A student looks for people, opportunities, and earlier work. A faculty member or alumnus maintains a research profile and can respond to a mentorship request. An administrator moderates records, manages accounts, and reads activity reports. The same business rules and the same database support every role, while each role sees a different set of screens. Separating the interface, the business rules, and the stored data is appropriate for this division of work.

1. Presentation Layer

The Presentation Layer is the user interface. It is built with React.js, HTML5, CSS3, and JavaScript. Screens are reusable React components, styled with plain CSS and the Manrope and DM Sans typefaces. Users open the platform in a modern web browser.

This layer provides the screens used for the platform’s main activities, including:

- Registration and sign-in
- Dashboards for students, faculty members, alumni, and administrators
- Faculty and alumni directories
- Research-area browsing
- Thesis and research opportunity listings
- Publication records, including conference papers entered as publication details
- The previous-research and thesis repository
- Research project records
- Mentorship requests and messages
- Notifications
- Search and filters
- Profile management, including a profile photograph
- Administrative moderation, account management, activity logs, and reports

A student, faculty member, or alumnus can upload a profile photograph from the computer. The accepted file types are JPEG, PNG, WEBP, and GIF, and the file may be at most 2 MB. A profile may instead keep an optional web address for a photograph. Administrator accounts do not have a profile photograph.

The Presentation Layer does not read or write the database. When a user acts, React sends the request to the Application Layer through the REST interface at */api/v1*. The project proposal identifies React.js for the user interface and a REST API for communication between that interface and the server.

2. Application Layer

The Application Layer is the processing layer of the system. It is implemented with Node.js and Express.js. It exposes the REST API at */api/v1* and applies the platform's rules before any data is stored or returned.

This layer manages the operations that give the platform its behaviour, including:

- Registration and sign-in, with passwords stored as bcrypt hashes and sessions issued as JSON Web Tokens
- Role-based access for student, faculty, alumni, and administrator accounts
- Profiles, departments, and research areas
- Thesis and research opportunities
- Publications, authors, and the previous-research repository
- Research projects and project members
- Mentorship requests, messages, and notifications
- Search and filtering
- Moderation of submitted records
- Account administration, including deletion of an account
- Activity logs and administrative reports

Students, faculty members, alumni, and administrators can create publication records and research projects. A conference paper is stored as a publication record: the conference name is kept as the venue, together with the title, abstract, publication year, and at least one author. The platform keeps this metadata and an optional web address. It does not upload a paper file, and it does not run manuscript review or conference management. An author or a project member must be an existing student, faculty member, or alumnus. The list used to choose that person returns the name, role, and department, and does not return an email address.

An administrator can delete another user's account. The signed-in administrator cannot delete that same account, and the last remaining administrator account cannot be deleted. Related mentorship records are removed with the account. A publication or project that still has another author or member is kept and assigned to that remaining person. Activity log entries written by the deleted user are retained.

For example, when a student submits a mentorship request, the React screen sends the request to Express. The API accepts the request only from an active student. It checks that the chosen mentor is an

active faculty member or alumnus, and it validates the subject and the message. If the request names a thesis opportunity, that opportunity must already be approved and open. PostgreSQL then stores the request and the first message, and the Application Layer records a notification for the mentor. The browser does not write to the database, and it does not notify the mentor by itself.

The project proposal identifies Node.js with Express.js for server-side processing, business rules, and the REST API that connects the interface to the data. Full manuscript submission and review, conference organisation, journal integration, payments, video sessions, and a mobile application remain outside the scope stated in the proposal.

3. Database Layer

The Database Layer stores, retrieves, and protects the platform's data. The database management system is PostgreSQL.

It holds the records the platform depends on, including:

- User accounts and roles
- Student, faculty, and alumni profiles
- Departments
- Research areas
- Publications and their authors
- Research projects and their members
- Thesis and research opportunities
- Mentorship requests and messages
- Notifications
- Activity logs
- Refresh tokens used to renew a signed-in session

Users do not connect to PostgreSQL themselves. Every read and every write passes through the Application Layer, which checks the caller's role and the state of the record before the database changes. Database constraints keep related records consistent. A person can be stored as a publication author or a project member only when that person has a student, faculty, or alumni profile. An approved publication or project is expected to keep at least one research area and at least one author or member.

The project proposal selects PostgreSQL because it is reliable, protects data integrity, and handles the structured records of a research platform: profiles, publications, opportunities, and mentorship.

Justification for Selecting Three-Tier Architecture

We selected Three-Tier Architecture for the following reasons:

1. Separation of concerns: Each layer has a specific responsibility. The user interface, the business rules, and the data remain logically separate. React renders screens. Express decides whether a request is allowed. PostgreSQL stores the result.

2. Maintainability: A change in one layer can be made with limited effect on the others. The visual design of a directory or a form can change without changing the database tables. A validation rule can change in the API without asking users to open the database.
3. Scalability: The architecture is a structured base for later research, reporting, or administrative features. New screens can call the same API, and new API rules can use the same database, without collapsing the three responsibilities into one program.
4. Security: Users do not open the database. Sign-in, role checks, and validation happen in the Application Layer before a query runs. Passwords are stored as bcrypt hashes. A session uses a short-lived access token and a separate refresh token. An uploaded profile photograph is accepted only after its type and size have been checked.
5. Modularity: Profiles, opportunities, publications, projects, mentorship, notifications, search, and administration are organised as modules behind the API. That organisation makes development and later maintenance easier than placing every rule inside the web pages.
6. Suitability for the technology stack: The architecture matches the technologies selected in the project proposal: React.js, HTML5, CSS3, and JavaScript → Node.js with Express.js → PostgreSQL.
7. Easier team development: The platform contains several modules and is being developed collaboratively by a four-member team. Separating the interface, the API, and the database lets members work on different parts in an organised way. Git and GitHub record those changes. The proposal organises the work across a ten-week schedule of requirement analysis, system design, database design, implementation, testing, deployment, documentation, and final presentation.

2. Methodology & Justify:

Selected Methodology: Agile Scrum

For BUP Research Bridge, we have selected the Agile Scrum methodology as the software development methodology. The project proposal adopts Agile Scrum because the platform has several modules and four kinds of users, and because the detail of a feature can be checked against the requirements while that feature is being built.

According to the proposal, Agile Scrum is selected to support iterative development, continuous feedback, flexibility, and team collaboration. The work is divided into smaller iterations, so the team can review what has been completed and decide the next piece of work. The proposal also states that weekly meetings will be held to review progress, discuss problems, and plan the next tasks, and that GitHub will be used for version control and collaboration. The description below follows that selection and the order in which the system was built.

How Agile Scrum Has Been Applied

The platform has been developed incrementally, rather than as one delivery of the entire system. Each iteration added a working part of Research Bridge. The iterations followed the order in which the system was constructed:

- Accounts, sign-in, and roles

- Profiles, departments, and research areas
- Thesis and research opportunities
- Publications and the previous-research repository
- Mentorship and notifications
- Search, moderation, and administrative reports
- Profile photographs and account administration

These parts belong to the requirements defined in the proposal: faculty and alumni profiles, thesis opportunities, publications and projects, the previous-research repository, search, authentication and role-based access, mentorship communication, notifications, activity logs, and administrative reports.

Key Practices of Agile Scrum for This Project

1. Iterative development: The system has been built through successive iterations. Each iteration delivered a defined set of functions, from accounts and profiles through opportunities, publications, mentorship, and administration.
2. Continuous testing: Testing has continued during development rather than being left until the end. The backend includes an automated test suite for the API. That suite runs against a separate test database, so a check of the application does not wait for a final testing week and does not depend on the development database.
3. Regular team meetings: The proposal specifies weekly meetings to discuss progress, identify problems, and plan upcoming work. That practice is part of the methodology selected for this project.
4. Continuous requirement validation: Features have been compared with the proposal's functional and non-functional requirements while they were being built. The aim is that the finished platform remains aligned with the needs of students, faculty members, alumni, and administrators.
5. Team collaboration: The four members collaborate through Git and GitHub, which the proposal names as the tools for version control and coordination. Work is divided by module, and changes are recorded in the shared repository.
6. Feedback and improvement: After an iteration, the result can be reviewed and the next iteration can close a gap. Later iterations added conference-paper entry on the existing publication record, allowed students to create publications and projects, added administrator account deletion, and added profile photographs.

Justification for Selecting Agile Scrum Methodology

1. Suitable for a multi-module system: The project contains directories, opportunities, publications, the thesis repository, mentorship, notifications, search, and administration. These modules can be developed one after another and still form one platform.
2. Supports refinement of requirements: When a requirement becomes more precise during development, it can enter a later iteration. A student could already be named as an author. Allowing students to create publications and projects, and recording a conference paper through the existing publication fields, were added without replacing the database design of those records.

3. Early testing and error detection: The API test suite, together with review during development, exposes defects before a final testing stage. A failure in sign-in, moderation, or mentorship can be corrected while the related module is still the focus of the work.
4. Better team coordination: The weekly meetings specified in the proposal, together with GitHub, coordinate the four members. Each member can see which module is in progress and which change has been recorded.
5. Earlier use of working features: Sign-in, profiles, opportunities, publications, and mentorship became usable as their iterations finished. The team did not have to wait until every administrative report and later account feature was complete before the earlier modules could be tried.
6. Appropriate for the project timeline: Section 8 of the proposal plans a ten-week schedule of requirement analysis, system design, implementation, testing, deployment, documentation, and final presentation. Agile Scrum organises those activities into manageable iterations instead of treating them as one uninterrupted build.